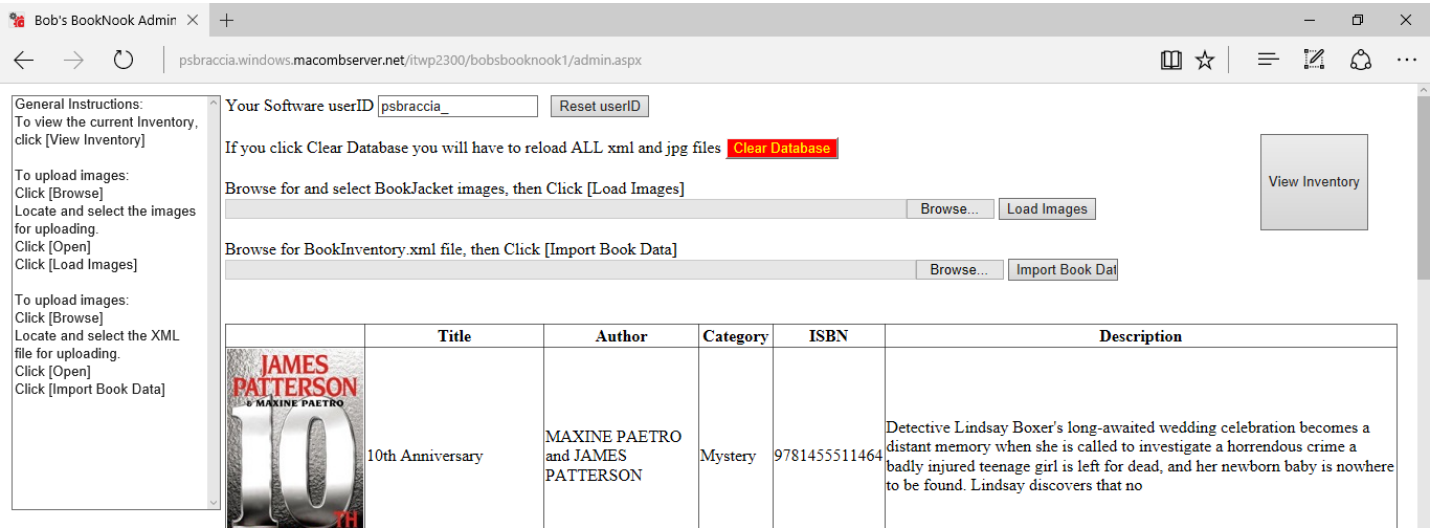
ITWP 2300

BUILDING DYNAMIC, INTELLIGENT WEB BASED SOLUTIONS WITH ASP.NET

PROJECT 1



In this unit, we begin the process of uploading information on the books in our inventory, including images representing jacket covers, ISBN, and author. Fortunately, this is an eBook store and every item sells for $5.99.

Page 15

FORWARD

While the looks and feel of this project are older in nature, it is not the presentation layer that we are as interested in as it is the database connection layer, product support, and data mining. We are interested in how the information for inventory control will be provided to us in a Business 2 Business (B2B) XML environment, what we must do to unravel the senders XML into a format that we can use in our database. As the project progresses, we want to know how to provide additional suggestions to the consumer to encourage extra sales, as well as when new materials come in, we want to alert our readers to books by authors that they have read in the past. You must understand that there are libraries available to do many of the tasks that are necessary for your project to work as intended. It is not necessarily prudent for me to simply say use this library or that library, but we want you to understand the mechanics behind the libraries so when something goes wrong, you will have the knowledge of the inner workings to fix any and all issues.

If I were to put this in terms of an automobile, I could say here are the keys, keep them in your pocket (on newer cars) and get behind the wheel. You have two peddles on the floor, the one on the right makes you go and the one on the left makes you stop. The wheel in front of you controls the directions. Now go.

OR

I could teach you how the engine provides power to the wheels to make you move, why you need to occasionally introduce gas, water, a new battery, what good is an oil change, lights, radio, seat adjustments, windshield wipers, and the many myriad points of the driving laws so that you understand what to do when operating a vehicle.

So is it with Bobs Book Nook. I want you to understand what is happening under the hood and behind the scenes.

INTRODUCTION:

The idea behind this project is straightforward. We want to enable data mining to encourage visitors to Bob's eBook Nook to buy more. It is an eBook store so that we do not have to worry about inventory levels and shipping. All books are $5.99.

Some of the objectives (besides the obvious what do we have in stock):

"People who purchased this book also purchased _____" to show up on our shopping cart page as an enticement.

We will send email to the subscriber whenever we add a new book by an author they have purchased in the past.

Book Nook E-Mail News!

Dear _____

We wanted you to know that we have a new title written by one of your favorite authors.
(image) _____ 's is now on sale!

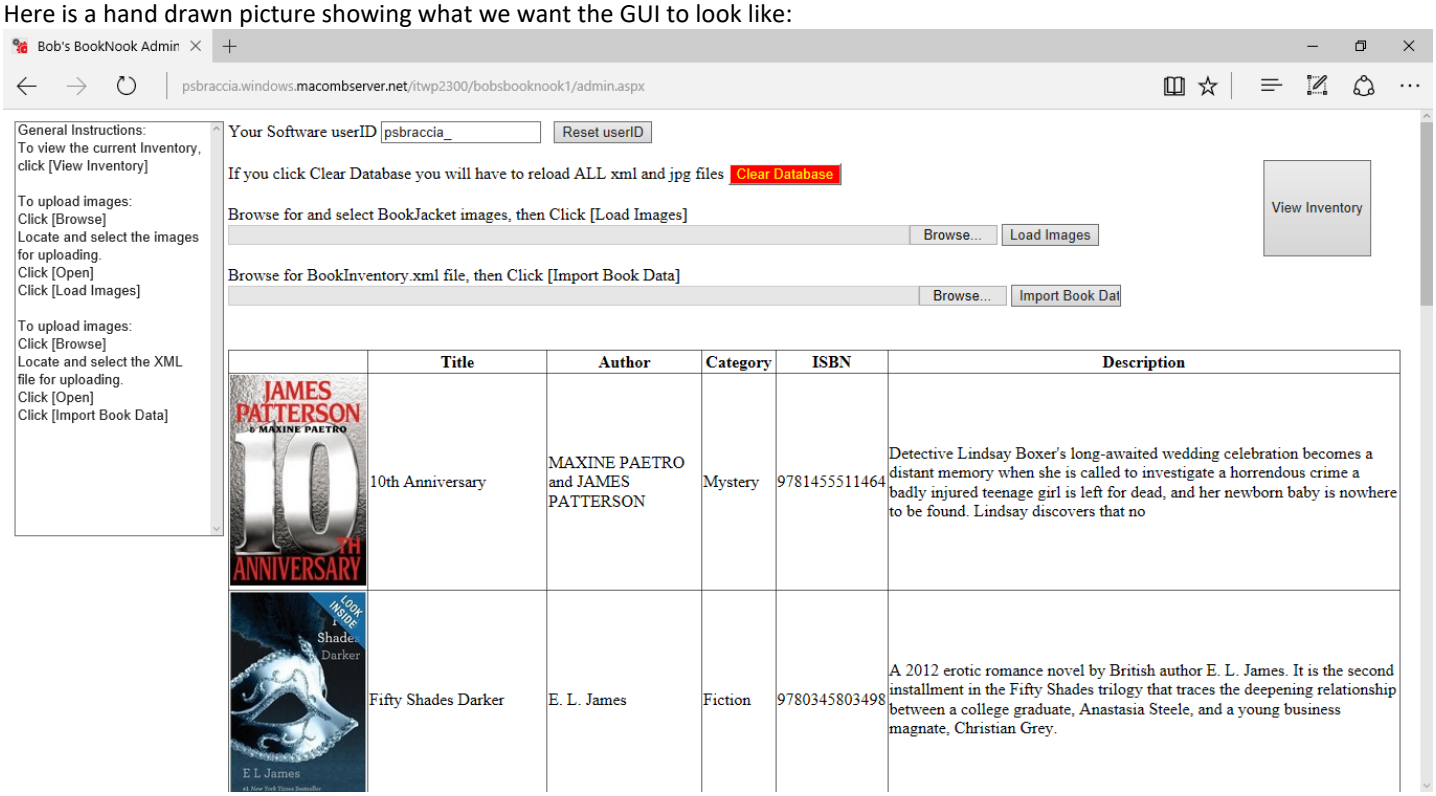
RESEARCH:

In discussions with the Project Manager, you learn the following facts:

- The first part of the project involves an ADMIN.aspx and ADMIN.aspx.cs page.
- You must re-use the same myDatabaseConnection.cs file you have been using.
- The data is being provided to the user from the publishers.
- The datafile is being emailed to the user in a zipped file format.
- The potential customer knows how to unzip the file to the desktop.

ITWP 2300 eBook Store Project 1

- The datafile contains:
  - An XML file titled BookInventory.xml
  - A folder titled BookImages containing .jpg images of the book jackets.
  - The XML file and the .jpg files must be stored in an **Img** folder on the server.
- Even though this particular project is for an individual customer, it will be marketed to other similar stores in the future. Because of this:
  - You will need to use a logon screen to capture the ownerID.
  - We will use a small file to maintain the ownerID.
  - Remember that the Database Manager requires that the ownerID end with an underscore.
- Because of the use of myDatabaseConnection.cs file, you must have a label for error messages as well as a gridview object to display the stored data.
- Because this project will eventually be "for anyone", you must make sure that any tables you need are available.
  - You can create a method that accepts the table name and the CREATE command to check for a table.
  - If a SELECT * FROM tablename returns a value, you do nothing.
  - If the SELECT * FROM tablename returns an error message, you create the table name.
  - [Homework 3].
- Here is a hand drawn picture showing what we want the GUI to look like:



- Any questions?
- The Clear Database button calls a Drop Table command? Yes.
- The two objects with what looks like a Browse button are fileupload controls? Yes.
- On the left, is that just a listbox? Yes. You can use it to indicate "progress" when uploading or downloading data. It contains some helpful hints upon startup.
- Does the grid view have to show up when the page is displayed? No, only when the View Inventory button is clicked.
- Please note you will be provided with the special code necessary to display the images. Research found a process that works on both the development computer and the production server as well.
- Will we be saving the images and the XML file in App_Data? No, you will place the images and XML file in an Img folder. This folder is located at the same hierarchical level as App_Data and App_Code.
- If we select the same images several times, will there be multiple copies in the Img folder? No. You must prevent duplicates via code.

- If we re-select the XML file, will it add duplicate records to the database? Yes, so you must invoke the UNIQUE keyword when creating your database table to keep out duplicates.
- Can we delete any records? Not at this point. We will look at that issue in a future meeting.
- Boss: "Just get this going this week. I expect a positive update by the standup meeting next Monday."

Wow, that might be a lot of information to digest all at once, but rest assured we will take things step by step. Fortunately, we have covered most all of the required techniques in recent homework assignments, so this should actually go quite well.

ITWP 2300 eBook Store Project 1

DESCRIPTION: Part 1

In a B2B relationship, that is Business to Business, information is shared through XML files. In this way, not sharing specific data layouts helps prevent some security issues. Each company can include whatever data it deems necessary for a particular transaction, leaving the implementation of that data to the individual businesses. In our case, book publishers have indicated that they will now ship Bob's Book Nook a zipped file containing an XML file and a zipped set of jacket images. They indicate that the XML file contains the following information on all published books: Title, ISBN, Author, Category, Image and Description. Knowing that these 6 items match our database requirements is a bonus.

What we will accomplish in part 1 is to create an **ADMIN.aspx** webpage which will allow us to pull the data from the publisher and upload all images to **Img** and upload the contents of the XML file to our database.

Create a folder on your desktop titled **"BookNookInventory**". After downloading the **BookInventory.**zip file from the eClassroom, place the **XML** file and the **bookImages** folder in the **newly created folder.**

Explanation: We need some place to stage the materials on our working desktop for us to be able to upload the new data to the working server, more specifically, we need to be able to place the image files in the **Img** folder. Yes, we know how to do it, but consider the admin from Bob's Book Nook. I am not sure I would grant them open access to App_Data, App_Code or Img! We need a way for your grandmother to be able to manage the information from the XML file and upload the images as well.

The first step is looking at the database to familiarize us with the data that the DBA wants to capture / display.

The Database Table Structure for Part 1

Table BOOKS:

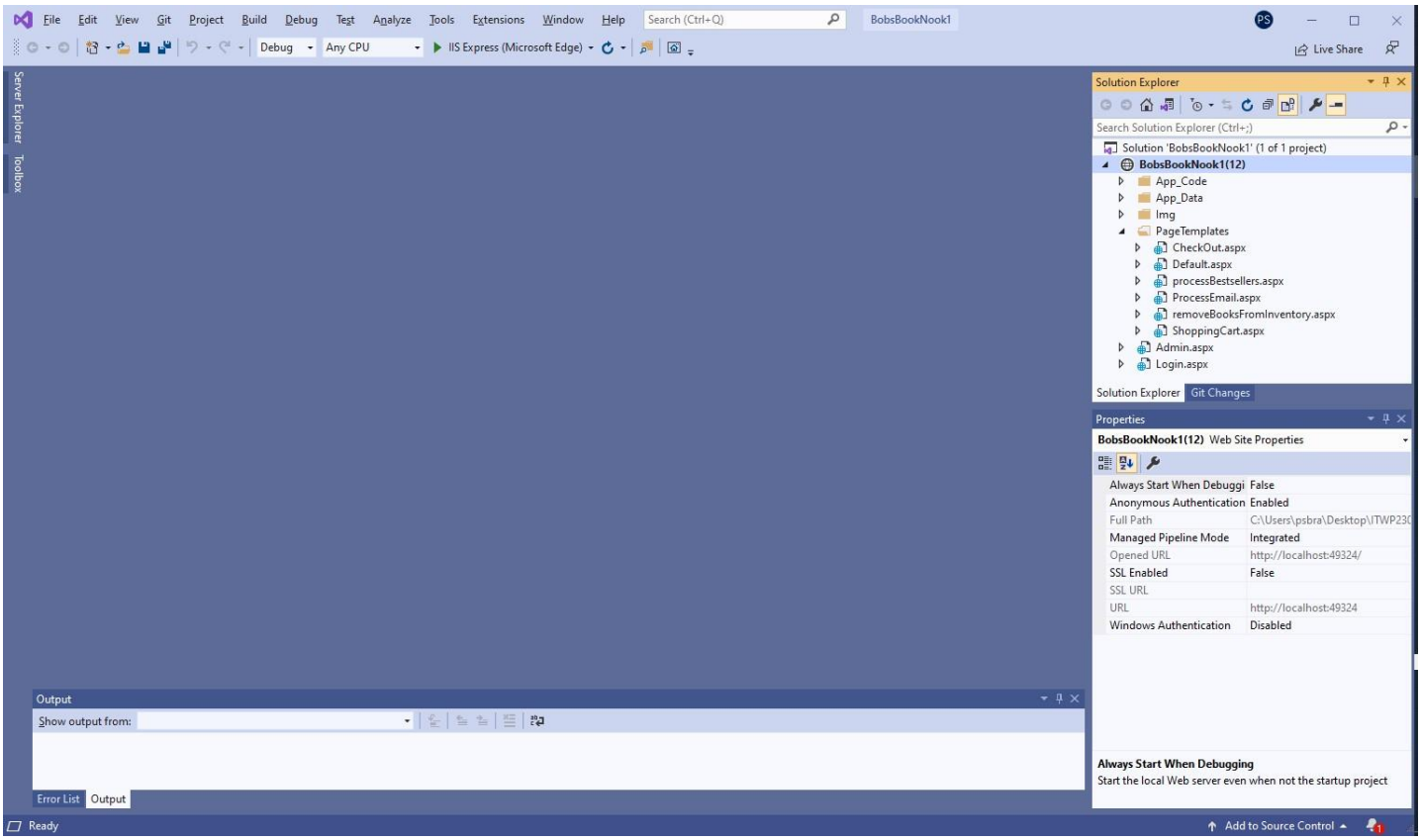
CATEGORY	JACKET URL	AUTHOR	TITLE	ISBN	DESCRIPTION

Note: The only table we will be using in Part 1 is the Books table.

PREPARATION:

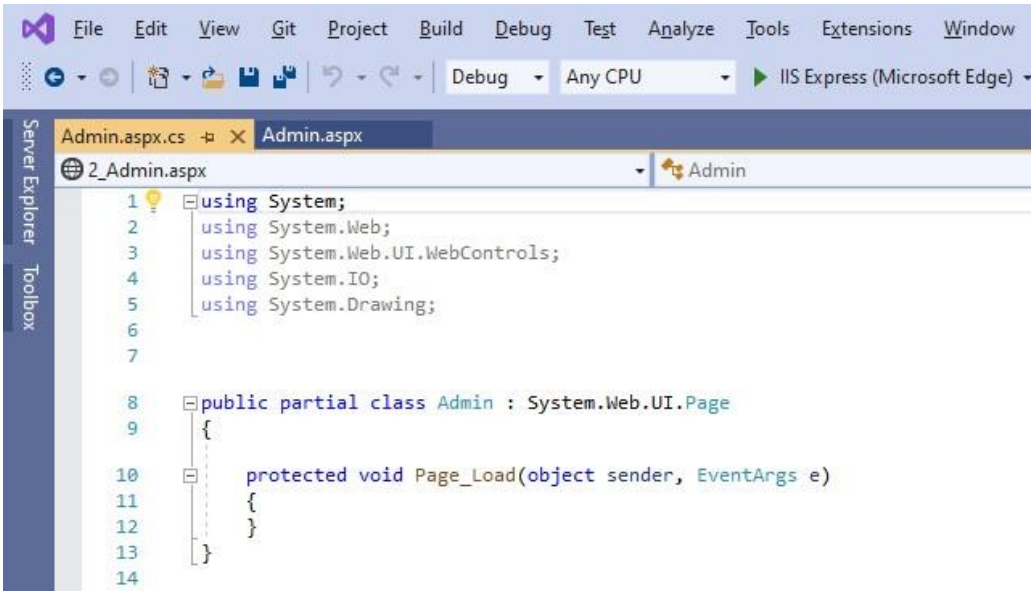
Download and unzip BookNookTemplate placing it within your ITWP2300 folder. Rename the folder **BobsBookNook1.**

You will find both an **App_Code** aspx and **App_Data** aspx folder and a new folder named **Img** as well as two Windows pages named **Admin.aspx** and **Login.aspx**.



Note: Pay particular attention to the project structure making sure Img is at the same level as App_Code. Notice that there is a folder named PageTemplates with other pages within which we will need to use as we progress thru the assignments.

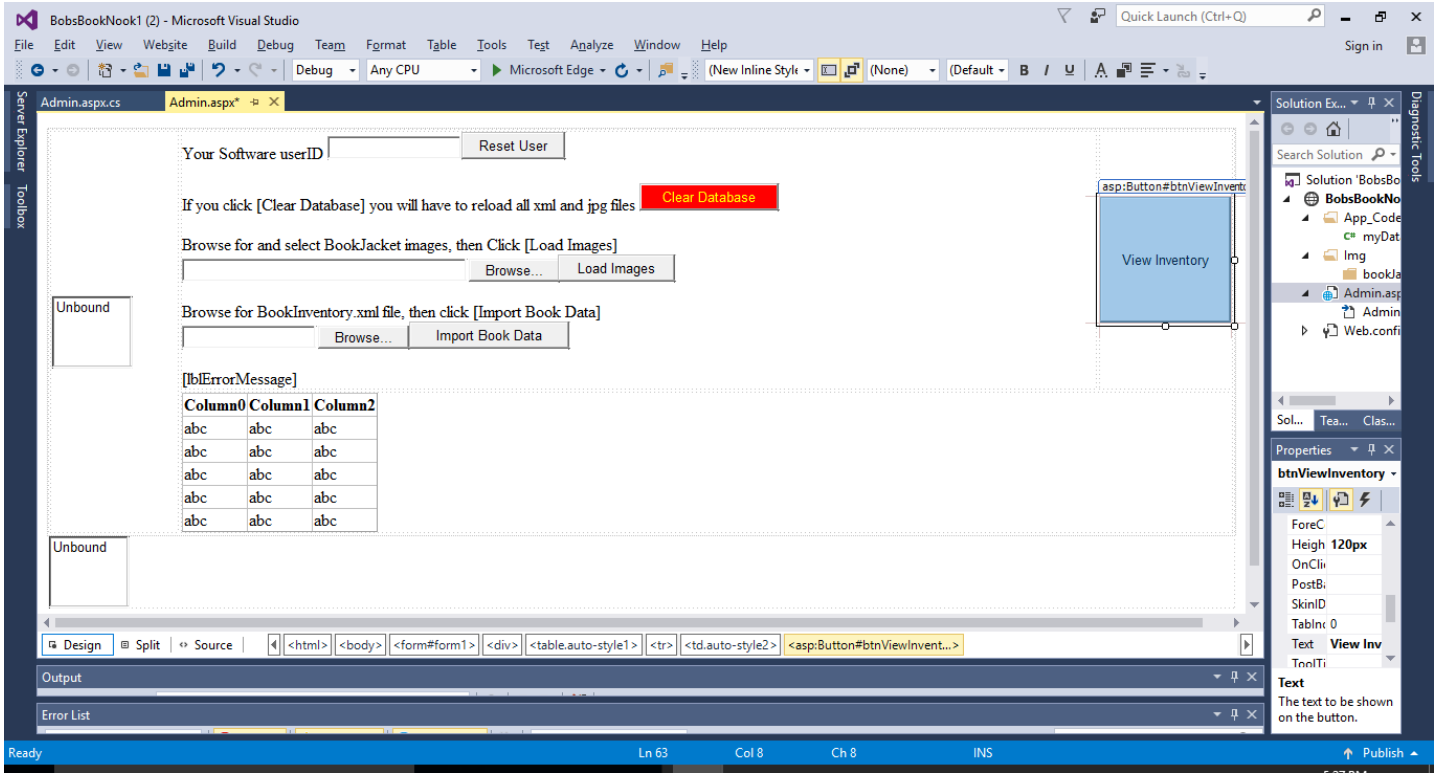
Taking a closer look at Admin,aspx.cs we see the using statements provided for us:



Even though they are greyed out which indicates that they are not currently used, it is just a temporary condition. It would be a good idea to validate prior to submission for grade.

On to the project:

- Copy myDatabaseConnection.cs from Homework 5 (Ashland Soccer) into App_Code.
- Set the title property of Login.aspx as appropriate.
- Set the title property of Admin.aspx Source page to Bob's BookNook Admin Page
- Right click on Admin.aspx in the Solution Explorer window select **Set As Start Page**.
- In Admin.aspx Design view, add a 2 x 3 table.
- In Column 2, Row 1 add the following:
    - The text 'Your Software OwnerID'
    - A TextBox named txtOwnerID with Enabled property set to false.
    - A button named btnResetUser. Text: Reset User
- Merge Column1, row 1 with column 1, row 2.
    - Insert a listbox named lstProgress in Column 1.
    - Make sure visible is set to true.
- According to the layout shown below:
    - The text If you click [Clear Database] you will have to reload all xml and jpg files
    - Replace [Clear Database] with a button named btnClearDB. Text: Clear Database. BackColor Red. ForeColor Yellow.
    - The text 'Browse for and select BookJacket images, then Click [Load Images]'.
    - Add a FileUpload object. Leave it named FileUpload1.
    - Add a button, btnLoadImages with text: Load Images.
    - The text 'Browse for BookInventory.xml file, then click [Import Book Data]'.
    - Add a FileUpload object. Leave it named FileUpload2.
    - Add a button, btnLoadXML with text: Import Book Data.
    - Add lblErrorMessage just below FileUpload2. Clear the Text value.
- In Column 3, Row 1
    - Add button btnViewInventory
    - Set text to View Inventory
- Merge Row 2, columns 2 and 3.
    - Insert a gridview object
    - Name it gvDisplay.
- Just under the table
    - Add a listbox lstHidden
    - Set it's visible property to true.
- Prior to turning in the project, reset the visible property to false.
- In Admin.aspx.cs add a Global variable named `string uploadPath;` This goes ABOVE Page_Load.



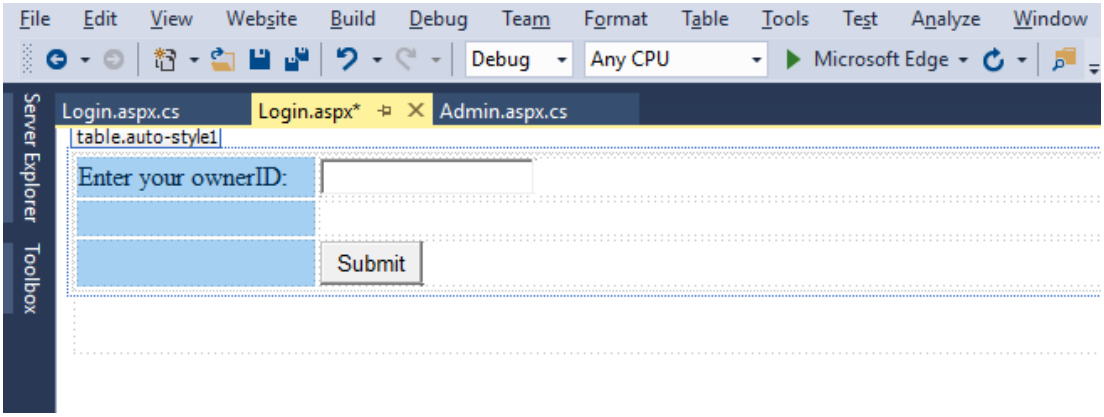
Note: We will repair the top alignment via CSS code in a few minutes.

Making Sure the Login Process Works.

Ed Note: While we did work with a login page earlier in the course, we will soon understand that sometimes it is better to grab only parts and pieces of pre-existing code and not to confuse the issue with too much stuff to remove and re-work. We can use the principle behind Login.aspx and Login.aspx.cs, but it is far more efficient in this instance to copy and paste just a few lines of code.

In **Login.aspx** Design view:

- Add 3 row by a 2 column table.
- In row 1, column 1, add "Enter your ownerID:".
- In row 1, column 2 insert a text box named txtOwnerID.
- In row 3, column 2 add a button named btnSubmit with a text value of Submit.
- Add **using System.IO.**



The code for Login.aspx.cs at this point is rather simple.

```
protected void btnSubmit_Click(…
{
   using (StreamWriter _writeFile = new StreamWriter(Server.MapPath("~/App_Data/ownerID.txt"), false))
   {
       _writeFile.WriteLine(txtOwnerID.Text); // Write the file.
       Response.Redirect("Admin.aspx");
   }
}
```

As soon as we know that the process works, we will code the associated "Business Rules".

Populate lbProgress

Working in **Admin.aspx.cs**, insert the following method call in Page_Load.

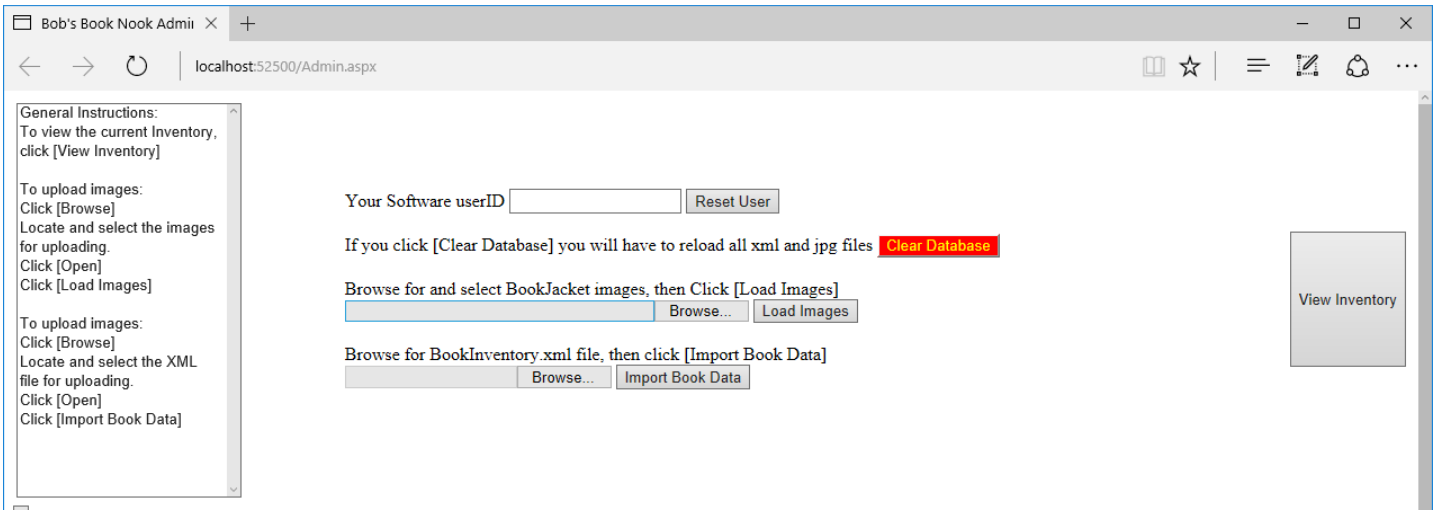
```
protected void Page_Load(…
    {
        initLstProgress();
    }
```

Create a method by right-clicking on the call statement.

Replace **`throw new NotImplementedException();`** with the following code in the newly created **initLstProgress()** method.

```
//throw new NotImplementedException();
lstProgress.Items.Clear();
lstProgress.Items.Add("General Instructions:");
lstProgress.Items.Add("To view the current Inventory,");
lstProgress.Items.Add("click [View Inventory]");
lstProgress.Items.Add("");
lstProgress.Items.Add("To upload images:");
lstProgress.Items.Add("Click [Browse]");
lstProgress.Items.Add("Locate and select the images");
lstProgress.Items.Add("for uploading.");
lstProgress.Items.Add("Click [Open]");
lstProgress.Items.Add("Click [Load Images]");
lstProgress.Items.Add("");
lstProgress.Items.Add("To upload images:");
lstProgress.Items.Add("Click [Browse]");
lstProgress.Items.Add("Locate and select the XML ");
lstProgress.Items.Add("file for uploading.");
lstProgress.Items.Add("Click [Open]");
lstProgress.Items.Add("Click [Import Book Data]");
lstProgress.Items.Add("");
```

Make sure your listbox is large enough to display the above data plus 5 or 6 lines at the bottom. (200 x 350 is a good start.)

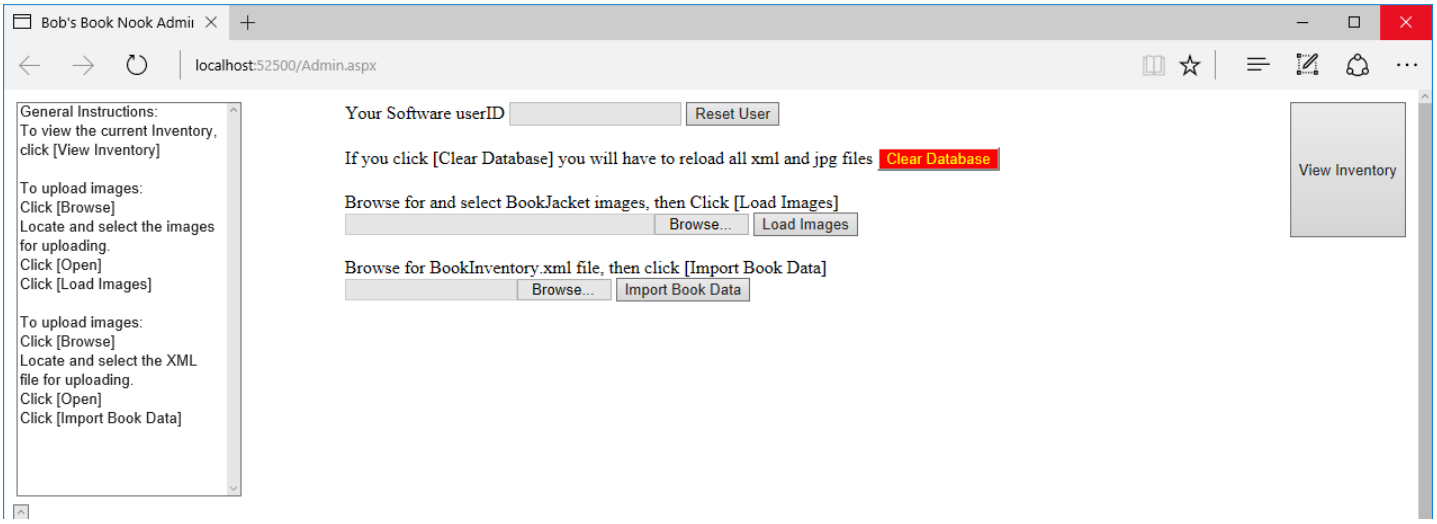


Fixing Vertical Alignment of All TD Elements.

Add the following code to your <Style> code block in Admin.aspx, Source view.

```
td {
        vertical-align: top ;
    }
```

Now all your text, buttons, and grid view will be top aligned.



Now That Our Page Is Designed, Let Us Continue with the OWNERID_!

In homework 4 we explored the use of a login screen and page redirection. We used cookies to maintain eMail and DOB as the key to the non-existent website. Back in Homework 2 we executed *psbraccia.windows.macombserver.net/dbtest* to see your id. We will be logging in with this id. MAKE SURE TO USE THIS USERID AS OTHER FILES WITH UNAPPROVED USERIDs WILL BE DELETED DURING SYSTEM MAINTENANCE.

The reason we need to implement any kind of login is to make sure that we can specify an **ownerID** which we will use in creating unique data tables in the database. You would not want Penny's Online Book Shop, Barns and Noble Online, or Bob to have to use the same ownerID! Each instance of the online book store will be assigned a unique ownerID. It just so happens we have 12 to 26 instances of the online bookstore, and the ownerID will be the ownerID given to you for the database. My online instance will use psbraccia_.

Business Rules

We used business rules to define the workings of the original Login code, and we will use business rules to define the modifications to Login.

Business Rule #1. ownerID can be any combination of letters, numbers, special characters, uppercase or lowercase. Must be validated.

Business Rule #2 The ownerID must end with an underscore.

Business Rule #3. The ownerID will be saved to a small file located in the **App_Data** folder. The name of this file is **ownerID.txt.**

Adding ownerID retrieval to Admin.aspx.

Make sure **Admin.aspx** is set as the Start Application.

Examine the Try Catch block of code we need to insert into Admin.aspx.cs/Page_Load.

```
protected void Page_Load(…
{

  try
        {

             var data = File.ReadAllText(Server.MapPath("~/App_Data/ownerID.txt"));
            //Using the Trim() function makes sure return and line feed codes are removed
             txtOwnerID.Text = data.ToString().Trim();

        } // end try
        catch
        {
            Response.Redirect("Login.aspx");
        }

    initLstProgress();

}
```

What we are looking to see is if the datafile is present in our implementation. If it is, we read the data and place it in the txtOwnerID text box. If it does not exist, we redirect the program to Login.aspx.

Click on the Reset User button and insert the code to simply redirect us to the login page.

```
protected void btnReset_Click(…
    {
         Response.Redirect("Login.aspx");
    }
```

Execute your program. Double check that exiting the program and restarting will reload the ownerID. Verify that the reset ownerID button works as expected.

*Note: If the Login screen always appears, highlight Admin.aspx, right click, select **Set As Start Page**.*

To implement the business rules we can add the following code to the submit button. It makes sense if we think about it. If everything is ok, redirect to Admin. Otherwise, stay in Login and fix the issues.

We need to populate row 2 in Login.aspx.

In row 2, column 1 add the following text: **Re-enter ownerID**
In row 2 column 2 insert a textbox named **txtOwnerID0**. Ed Note: You can copy txtOwnerID.

```
protected void btnSubmit_Click(…
{
    //use a bool flag to indicate errors
    bool okFlag = true;
    //make sure ownerID is entered twice in the form
    if ((txtOwnerID.Text == "") || (txtOwnerID0.Text == ""))
    {
        okFlag = false;
        txtOwnerID.Text = "";
        txtOwnerID0.Text = "";
        txtOwnerID.Focus();
    }
    //make sure ownerIDs match
    if (txtOwnerID.Text != txtOwnerID0.Text)
    {
        okFlag = false;
        txtOwnerID.Text = "";
        txtOwnerID0.Text = "";
        txtOwnerID.Focus();
    }
    //if ownerID does not end with underscore, simply add it.
    if (!txtOwnerID.Text.EndsWith("_"))
    {
        txtOwnerID.Text += "_";
    }
    //Check okFlag for 'permission' to go back to Admin.
    if (okFlag)
    {
   // this is the original code to write the file and move to Admin.
   using (StreamWriter _writeFile = new StreamWriter(Server.MapPath("~/App_Data/ownerID.txt"), false))
        {
            _writeFile.WriteLine(txtOwnerID.Text); // Write the file.
            Response.Redirect("Admin.aspx");
        }
    } //end of if(okFlag)
}
```

Magic in Admin Page_Load

For this section, switch to Admin.aspx.cs.

There are only three more commands that need to be added to Page_Load. They are inserted just after the ownerID code and before the call to initLstProgress, shown below.

```
try
{
```

```
   var data = File.ReadAllText(Server.MapPath("~/App_Data/ownerID.txt"));
   txtOwnerID.Text = data.ToString().Trim();
} // end try
catch
{
   Response.Redirect("Login.aspx");
}

//not necessary for FileUpload2
//allows us to select all jpg files at the same time.
FileUpload1.AllowMultiple = true;
//Set path to Img
uploadPath = Server.MapPath("~/Img/");
//used when creating a new book store.
//The following statement is on one line of code.
checkTables(txtOwnerID.Text + "BOOKS", "CREATE TABLE " + txtOwnerID.Text + "BOOKS (Title
CHAR(64), ISBN CHAR(14) UNIQUE, Author CHAR(64), Category CHAR(48), Image CHAR(128),
Description CHAR(255))");

initLstProgress();
```

You will see an error indicated for checkTables. Go ahead and create the method.

Before we delve into checkTables: **FileUpload1.AllowMultiple = true;** is used to enable us to select multiple files. In our case, we can select all of the jpg files sent to us by the publisher. We do not need to do this for **FileUpload2** as we are only looking for a single XML file.

We have used MapPath in the past to indicate where data files are located. In this assignment, we need to point to the Img folder on the production server.

When creating **checkTables** you will see that we are providing two options. The name of the table, and the CREATE TABLE command necessary if the table does not exist. We did similar actions in Homework 3. We could actually add checkTables to myDatabaseConnection as this code snippet is perfect for OOP. We would have to modify the current code to include ref gvDisplay and ref lblErrorMessage. It is left to you to implement this as a member of myDataBaseConnection.cs for Extra Credit. Please be sure to indicate in the notes section of the assignment page in the eClassroom that you were able to move the code.

checkTables

The GUI created the following code block for you.

```
private void checkTables(string v1, string v2)
    {
        //throw new NotImplementedException();
    }
```

String v1 and string v2 do not make much sense, so let's rename them according to their use:

```
private void checkTables(string Table, string sqlStatement)
{
    //throw new NotImplementedException();
}
```

As we did back in Homework 3, we need to see if an error is returned when we execute a SELECT * FROM *Table*.

If there is an error reported, we then call the myDatabaseConnection.ExecuteSQL to create the table.

```
private void checkTables(string Table, string sqlStatement)
{
    //throw new NotImplementedException();
    lblErrorMessage.Text = "";

    myDatabaseConnection.executeSQL("SELECT * FROM " + Table, ref gvDisplay, ref
lblErrorMessage);

    //turn off gridview
    gvDisplay.Visible = false;
    //check for error message: create the connection if necessary
    if (lblErrorMessage.Text.Length > 0)
    {
        lblErrorMessage.Text = "";
        myDatabaseConnection.executeSQL(sqlStatement, ref gvDisplay, ref
lblErrorMessage);
    }
}
```

This code needs little further explanation as it is taken directly from Homework 3.

Note: We use this specifically because of a new deployment situation where no database tables exist for the client. We will be using this to create the other tables that we will need as we build our store.

Let's take the remaining buttons from the top down.

Clear Database

Click on Clear Database button and enter the following code.

```
protected void btnClearDatabase_Click(…
    {
        string sqlCommand = "Drop Table " + txtOwnerID.Text + "BOOKS";
        myDatabaseConnection.executeSQL(sqlCommand, ref gvDisplay, ref lblErrorMessage);
    }
```

As you can see we simply use the drop command to clear the database.

FileUpload1 and FileUpload2

No code required.

btnLoadImages

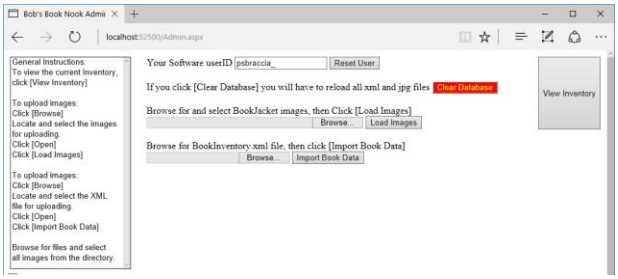
We will use the FileUpload HasFiles attribute to see if we have selected one or more files. Unfortunately, web information on how to set the search extension of the FileLoader object is incomplete at best. Under the circumstances, we can make sure jpg files have been selected, but we cannot exclude all other file types.

Ed Note: For the multi-file control to work, you must make sure you are in .NET 4.6.1.

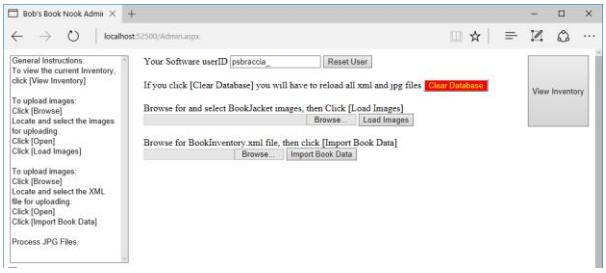
How? WebSite -> Start Options -> Build
Check .Net Version.

```
protected void btnLoadImages_Click(…
    {
        //Clear list and label
        lstHidden.Items.Clear();

        //check to see if you have selected a file
        if (!FileUpload1.HasFiles)
        {
            //if not, error message is displayed and focus set to fileupload
            lstProgress.Items.Add("Browse for files and select");
            lstProgress.Items.Add("all images from the directory.");
            FileUpload1.Focus();
        }
        else
        {
            lstProgress.Items.Add("Process JPG Files.");
        }
    }
```



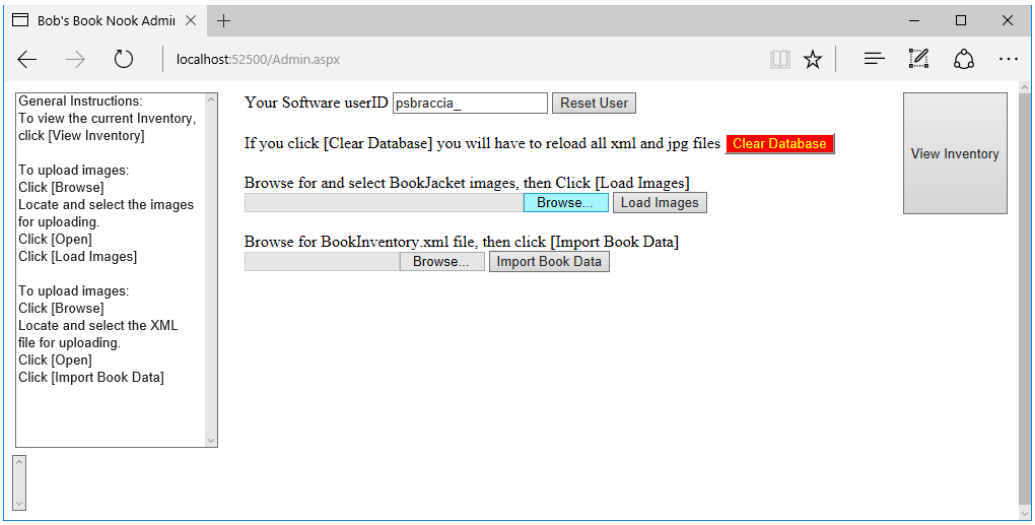
In the figure above you can see the message added to the bottom of lstProgress with focus being placed on FileUpload1, specifically the Browse button.



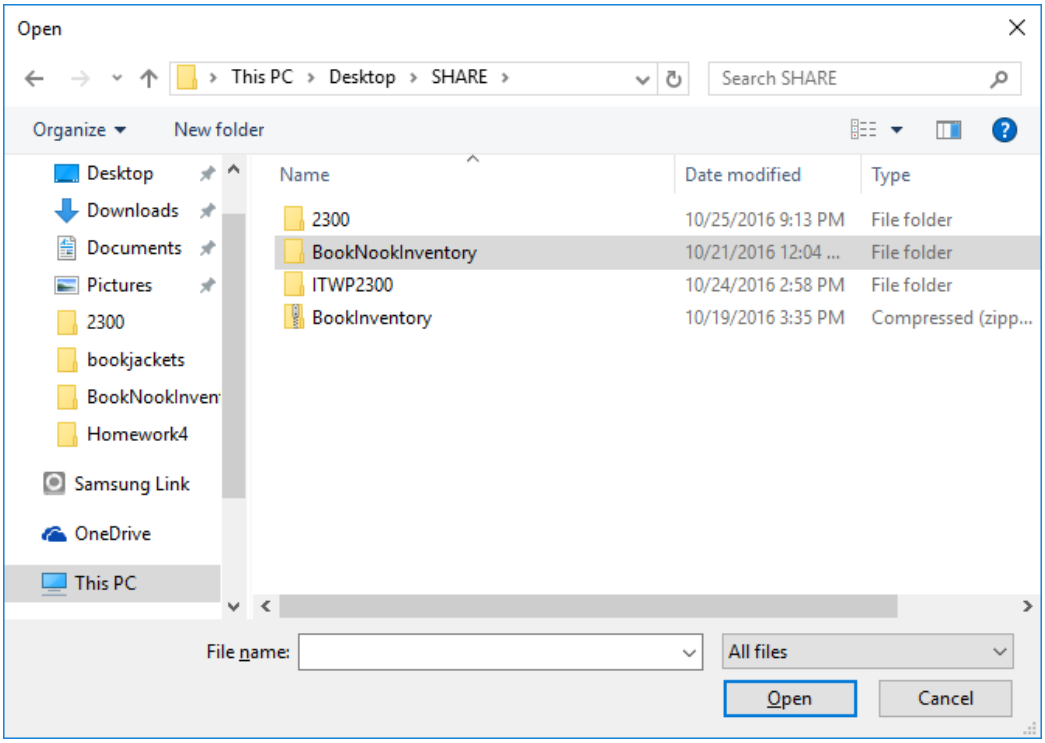
In the figure above, we selected some images and were “told” to process JPG files.
It may be prudent to take a few moments and explore the “Users Manual” for jpg upload.

EXCERPT FROM THE USER'S MANUAL

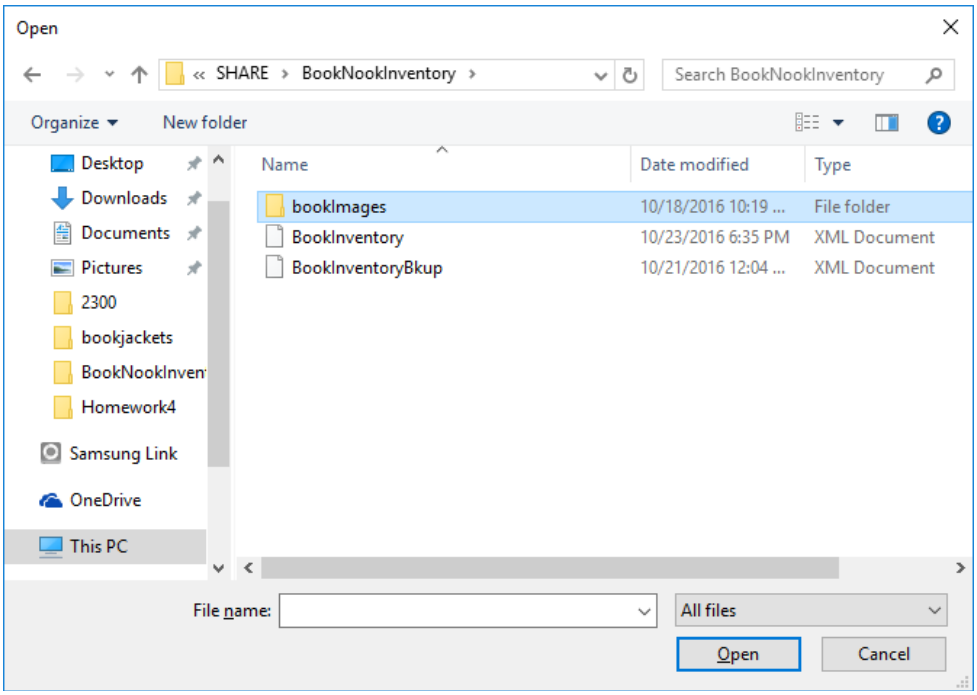
How to upload jpg images from the publisher.



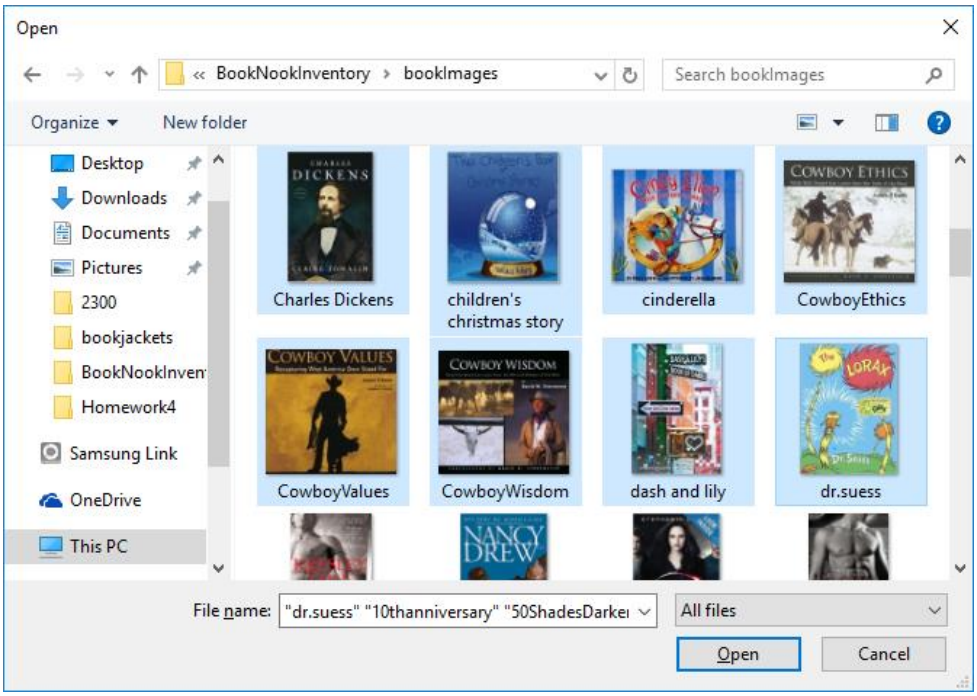
Select the first Browse Button next to [Load Images]



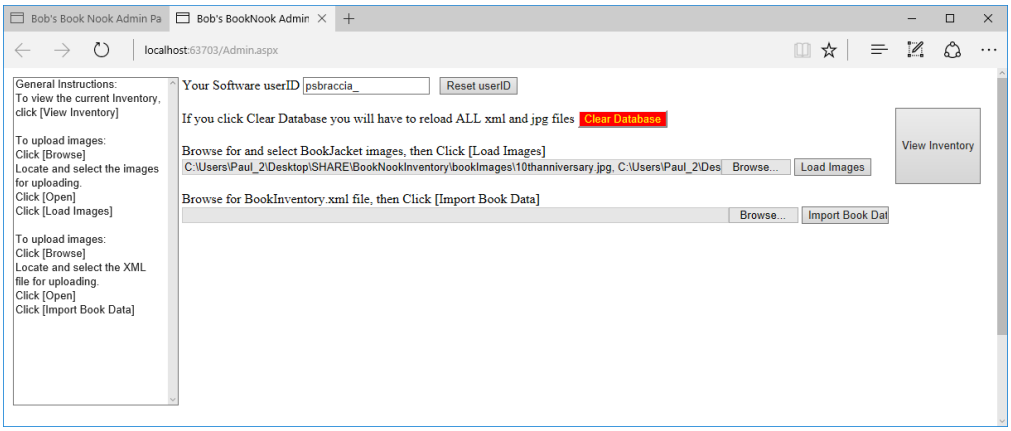
Locate the BookNookInventory folder that you downloaded from the Publisher.
Double click to open the folder.



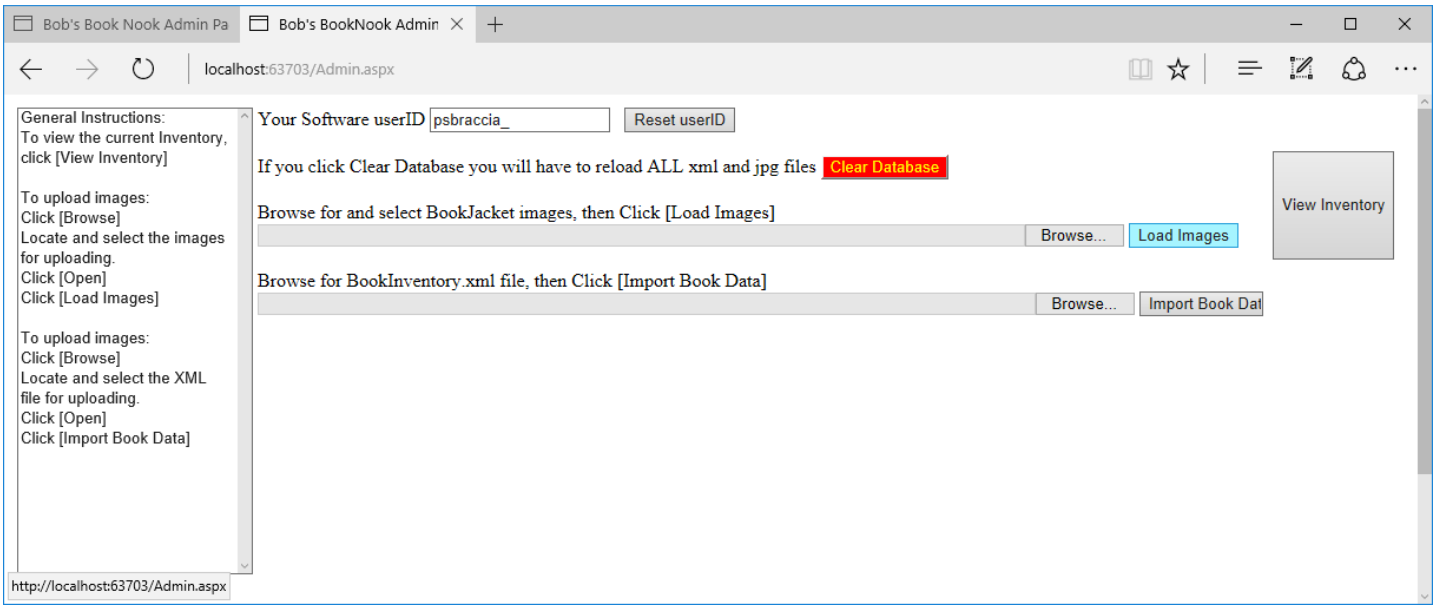
Locate and open bookImages.



Select all or some of the images.
Then click [Open] once you have made your selection.



A number of file names will appear in the box to the left of [Load Images].
You may now click on [Load Images] to upload the bookjacket icons to the Web.



The box of filenames will clear indicating a successful upload.

We now return you to the text.

btnLoadImages cont.

Within the else statement, we add the following bold code.

```
else
{
    lstProgress.Items.Add("Process JPG Files.");

     foreach (HttpPostedFile uploadedFile in FileUpload1.PostedFiles)
     {
        //if the file exists on the server, it will simply be overwritten.
        string fileName = Path.GetFileName(uploadedFile.FileName);
        uploadedFile.SaveAs(Server.MapPath("~/Img/") + fileName);
        lstHidden.Items.Add(String.Format("{0}", fileName));
     }
}
```

Let's take this one line at a time.

```
foreach (HttpPostedFile uploadedFile in FileUpload1.PostedFiles)
    {
```

We can separate the list of filenames storing them in uploadedFile. The foreach allows us to process each file in turn.

```
      string fileName = Path.GetFileName(uploadedFile.FileName);
```

Consider that our upload path was d:\\someServer\\http:\\Img\\ and the file name was something like c:\\mydesktop\\homework\\booknookInventory\\bookjackets\\orsonWells1.jpg.

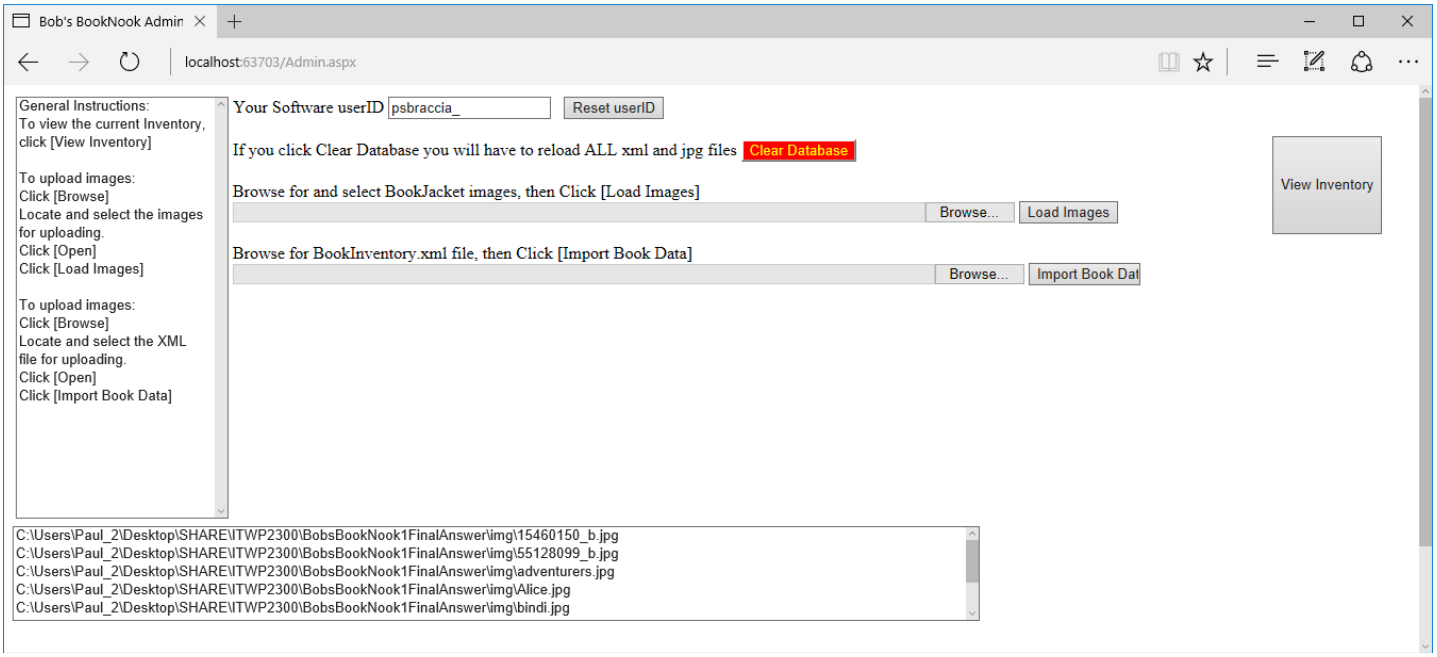
The results of this code would create a fileName of d:\\someServer\\http:\\Img\\orsonWells1.jpg.

```
   uploadedFile.SaveAs(Server.MapPath("~/Img/") + fileName);
```

The file is uploaded using the SaveAs function.

```
 lstHidden.Items.Add(String.Format("{0}", fileName));
```

The user will never see this list, but we can use it during debugging. Just make it visible and very very wide.



You may want to consider adding a "*" to lstProgress to indicate that the process is progressing.

Ed Note: It is understood that we do not check that the data type is JPG. It is not hard to add, and is left to you to implement.

I would be made to think that we would have to dissect the extension (lastIndexOf(".")) and add an additional if statement that wraps around the if statement which excludes any non jpg file.

You could include it in the current if statement like this:

```
//extract the extension into string ext.
//using LastIndexOf
if (!File.Exists(fileName) && (ext == "jpg"))
{
//Code to skip the file upload
}
else
{
//original upload code here
}
```

Import Book Data

```
protected void btnLoadXML_Click(…
{
    lstHidden.Items.Clear();
    lblErrorMessage.Text = null;
    //check to see if you have selected a file
    if (!FileUpload2.HasFile)
    {
        //if not, error message is displayed and focus set to fileupload
        lstProgress.Items.Add("[Browse] for the XML file");
        lstProgress.Items.Add("and select.");
        FileUpload2.Focus();
    }
    else
```

```
    {
        lstProgress.Items.Add("Process XML File.");
    }
}
```

If this code looks familiar, it is basically what we started the jpg file upload method with.

If FileUpload2 has no file, post an error else write a “progress” message.

The direction the development staff took on this project, was to save the XML file to the Img folder where we could then read the data as we did in Homework 1 and process the XML schema.

Let’s save the file first, then we will take a closer look.

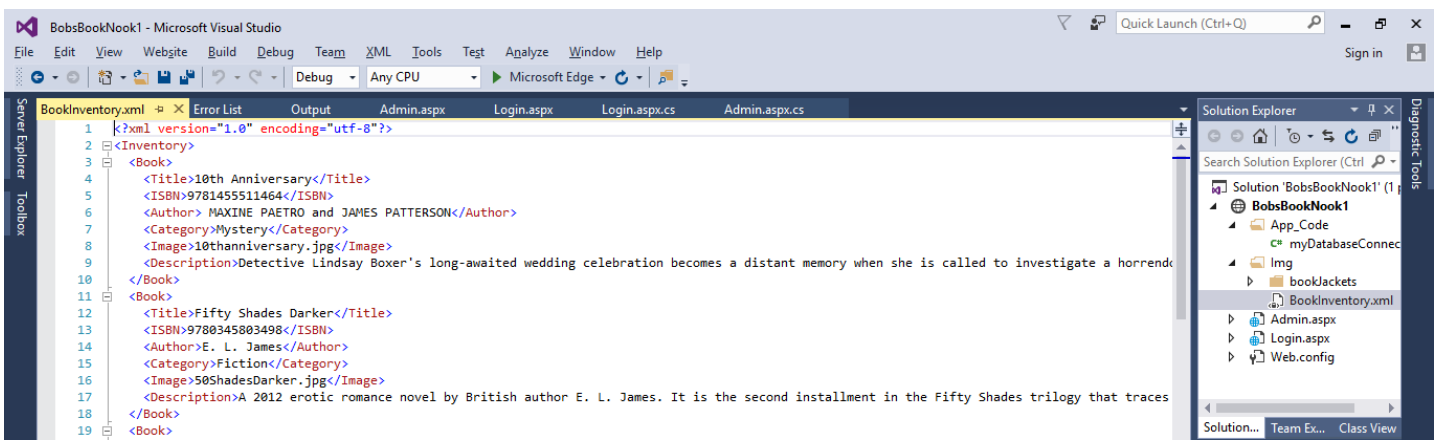
We expand the else statement as shown below.

```
else
{
    lstProgress.Items.Add("Process XML Files.");

    String fileName = uploadPath + FileUpload2.FileName;
    lstProgress.Items.Add(fileName);
    FileUpload2.SaveAs(fileName);

}
```

You can validate that the upload occurred by taking a peek in the Img folder.



What is in that XML File?

```
<?xml version="1.0" encoding="utf-8"?>
<Inventory>
    <Book>
      <Title></Title>
      <ISBN></ISBN>
      <Author></Author>
      <Category></Category>
      <Image></Image>
      <Description></Description>
  </Book>
</Inventory>
```

As you can see the file starts with a version tag.
The Inventory tag marks the beginning and end of the file.
The 2nd level Book tag (consider it a Record) is repeated for each text in the entire inventory.
The 3rd level tags hold the fields for the record Book.

```
<?xml version="1.0" encoding="utf-8"?>
<Inventory>
  <Book>
    <Title>baseball A History of the National Pastime</Title>
    <ISBN>1546015098543</ISBN>
    <Author>Mark Stewart</Author>
    <Category>Sports</Category>
    <Image>15460150_b.jpg</Image>
    <Description>A comprehensive history of baseball focusing on its evolution,
momentous events, and key personalities</Description>
  </Book>
  <Book>
    <Title>Heroes of Baseball: The Men Who Made It America's Favorite
Game</Title>
    <ISBN>9873432156789</ISBN>
    <Author>Robert Lypsyte</Author>
    <Category>Sports</Category>
    <Image>55128099_b.jpg</Image>
    <Description>Their names echo through the halls of time and the National
Baseball Hall of Fame.</Description>
  </Book>
</Inventory>
```

As we did in Homework 2, it is our task to read the datafile, parse the fields by removing the <tags> and create an acceptable SQL INSERT command.

Reading the XML File from the IMG Folder

Add the bolded code below to btnLoadXML, specifically in the else block of code.

```
else
{
    lstProgress.Items.Add("Process XML Files.");
    String fileName = uploadPath + FileUpload2.FileName;
    lstProgress.Items.Add(fileName);
    FileUpload2.SaveAs(fileName);
    //Read xml file to lstHidden
     try
     {
         if (File.Exists(fileName))
             using (StreamReader sr = new StreamReader(fileName))
             {
                 while (sr.Peek() >= 0)
                 {
                     lstHidden.Items.Add(sr.ReadLine());
                 }
             }
     } // end try
```

```
    catch (Exception f)
    {
        Console.WriteLine("The process failed: {0}", f.ToString());
    }
}
```

At this point you should understand the code. Realize that each record in the XML file will be placed in lstHidden for us to process.

```
<?xml version="1.0" encoding="utf-8"?>
<Inventory>
<Book>
<Title>10th Anniversary</Title>
<ISBN>9781455511464</ISBN>
<Author> MAXINE PAETRO and JAMES PATTERSON</Author>
<Category>Mystery</Category>
<Image>10thanniversary.jpg</Image>
<Description>Detective Lindsay Boxer's long-awaited wedding celebration becomes a distant memory when she is called to investigate a horrendous crime a badly
```

Rather than making this method more complicated, we will call a separate method to unscramble things.

decode(lstHidden);

This call statement is the last statement in btnLoadXML_click. //The call to decode must be placed within the else code block.

```
        decode(lstHidden);
        }//end else
    }//end method
}//end class
```

Generating the method decode() gives us this script.

```
  private void decode(ListBox lstHidden)
    {
        // throw new NotImplementedException();
    }
```

Add the following bold code.

```
private void decode(ListBox lstHidden)
    {
        // throw new NotImplementedException();

        //method level variable
        string field;

        //remove the first two lines from lstHidden
        //we do not need the <?xml version="1.0" encoding="utf-8"?>
        //nor do we need to process < Inventory >

         lstHidden.Items.RemoveAt(0);
         lstHidden.Items.RemoveAt(0);

    }
```

```
Process XML Files.
C:\Users\Paul_2\Desktop\SHAF

<Book>
<Title>10th Anniversary</Title>
<ISBN>9781455511464</ISBN>
<Author> MAXINE PAETRO and JAMES PATTERSON</Author>
<Category>Mystery</Category>
<Image>10thanniversary.jpg</Image>
<Description>Detective Lindsay Boxer's long-awaited wedding celebration becomes a distant memory when she is called to investigate a horrendous crime a badly
</Book>
```

Notice that the first tag is now the <Book> tag.

Presented herein is the entire decode method however stripTags method is delayed.

```
private void decode(ListBox lstHidden)
    {
        // throw new NotImplementedException();

        //method level variable
        string field;

        //remove the first two lines from lstHidden
        //we do not need the <?xml version="1.0" encoding="utf-8"?>
        //nor do we need to process < Inventory >

        lstHidden.Items.RemoveAt(0);
        lstHidden.Items.RemoveAt(0);


        //Create the start of an SQL statement
         while (lstHidden.Items.Count > 1)
         {
             string sqlStatement = "INSERT INTO " + txtOwnerID.Text + "BOOKS (Title,
ISBN, Author, Category, Image, Description) VALUES (";

             //process 6 tags
             //remove <books> tag
             lstHidden.Items.RemoveAt(0);

             for (int i = 0; i < 6; i++)
             {
                 //pull the top record from lstHidden
                 field = lstHidden.Items[0].ToString();
                 //call stripTags to remove each third level matching tag.
                 field = stripTags(field);
                 //field 5, i == 4, is the jpg field
                 //We are just saving the folder and file name
                 if (i == 4)
                 {
                     sqlStatement += "'" + "\\Img\\" + field + "'";
                 }
                 else
                 {
                     // if description is over 255 characters, it needs to be truncated
to fit our field descriptors
                     if (i == 5) //255 length
                     {
```

```
                    if (field.Length >= 255)
                    {
                        field = field.Substring(0, 254);
                    }
                }
                //replace all ' with  '':  single with double
                field = field.Replace("'", "''");
                //append to sqlStatement
                sqlStatement += " '" + field + "'";
            }
            if (i < 5) //5 is the last field.  We are not yet there, so add a ,
            {
                sqlStatement += ", ";
            }
            //remove record from lstHidden
            lstHidden.Items.RemoveAt(0);
        }
        //when all 6 records have been processed, append the closing )
        sqlStatement += (")");
        //see what it looks like on the screen
        Response.Write(sqlStatement + "<hr>");
        //myDatabaseConnection.executeSQL(sqlStatement, ref gvDisplay, ref
lblErrorMessage);
        //remove </books> tag
        // Make sure this is within the while code block.

        lstHidden.Items.RemoveAt(0);
    }
}
```

Note: The above code is commented enough so that you will be able to understand the intent.

Now for that stripTags method.

As you see in the code above, **field = stripTags(field);** is used to call stripTags. The purpose of stripTags is to remove the leading and trailing field (3rd level) tags returning just the contents.

Here is what happens to the data step by step.

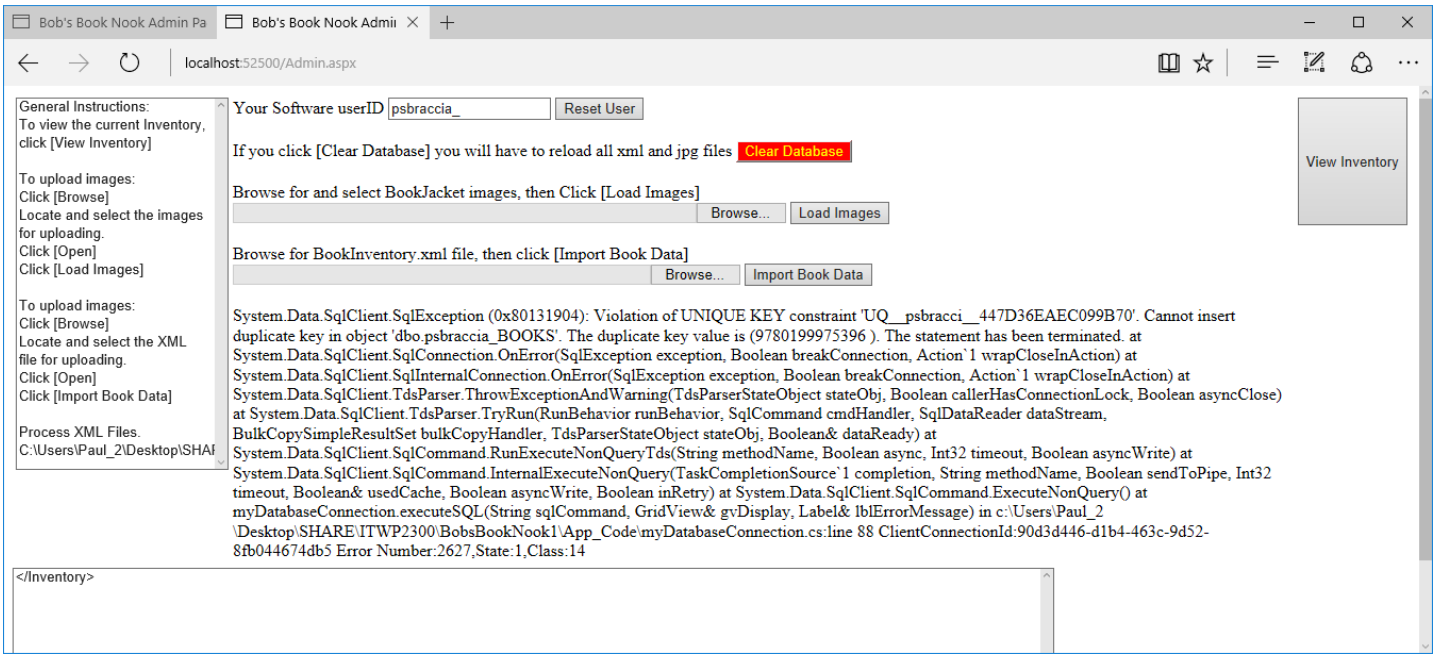
- field = <Title>Gone With The Wind</Title>
- Send field to stripTags.
- Locate the > and store the position in x.
- x = 6
- strip off the first 7 characters and store the string back in field.
- field = Gone With The Wind</Title>
- Locate the last occurrence of < and set the positional value in x.
- x = 18
- Save only the first part of the field up to but not including the <
- field = Gone With The Wind
- field is returned to the calling statement and placed in the local instance of field.

Here is the code for stripTags.

```
private string stripTags(string field)
{
    // throw new NotImplementedException();
    //remove up to first >
    int x = field.IndexOf(">");
    field = field.Substring(x + 1);
    //remove second tag
    x = field.LastIndexOf("<");
    field = field.Substring(0, x);
    return (field);
}
```

Once the sql statements look like they should, comment out the Response.Write statement and uncomment the call to myDatabaseCommand.executeSQL(…

If you get something that looks like this:



Just [Clear Database].

View Inventory is just a bit more complicated than select * from Table. We are going to have to interact with the gridview design, both in the actual gvDisplay object, and in the source page of Admin.aspx.

Take a break, and when we return, it is View Inventory!

View Inventory

Add the following function to Admin.aspx.cs.

```
    protected string FormatImageUrl(string url)
    {
        if (url != null && url.Length > 0)
            return ("~/" + url);
        else return null;
    }
```

Double click btnViewInventory.

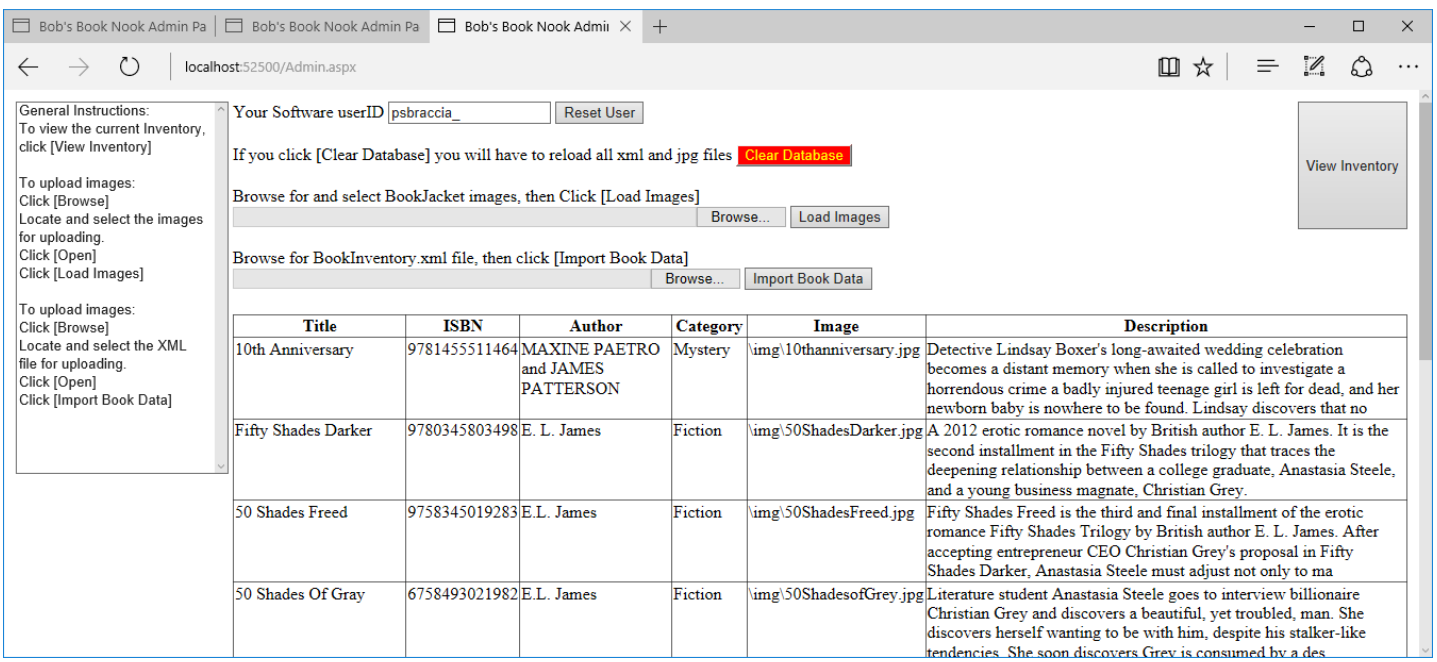
```
protected void btnViewInventory_Click(…
    {

    }
```

Add the following code.

```
protected void btnViewInventory_Click(object sender, EventArgs e)
{
    string sqlCommand = "SELECT Title, ISBN, Author, Category, Image, Description FROM "
+ txtOwnerID.Text + "BOOKS";
    myDatabaseConnection.executeSQL(sqlCommand, ref gvDisplay, ref lblErrorMessage);
}
```

Running your app at this point, you see all the fields.



We will modify the grid view to re-organize the layout of the data.

We could select the gridview object, click on the right arrow, edit columns, and hope everything works out.

Fortunately, we can make edits to the gridview object in Source view of Admin.aspx.

Look for the gridview object in Admin.aspx[.

```
<asp:GridView ID="gvDisplay" runat="server" >

</asp:GridView>
```

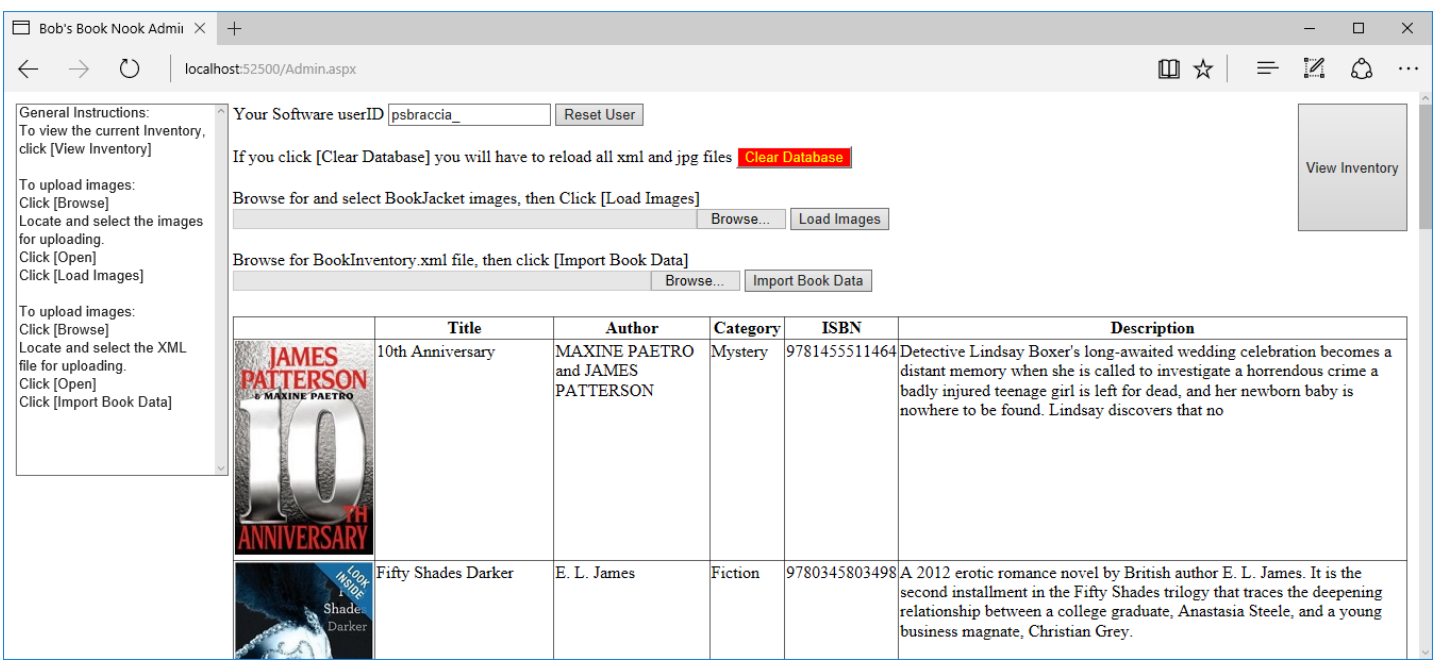
Replace the original GridView tags with the following HTML code.

```
<asp:GridView ID="gvDisplay" runat="server" AutoGenerateColumns="False">
<Columns>
 <asp:TemplateField>
  <ItemTemplate>
<asp:Image ID="Image1" runat ="server" ImageUrl='<%# (string) FormatImageUrl( (string) Eval("Image")) %>' />
  </ItemTemplate>
 </asp:TemplateField>
 <asp:BoundField DataField="Title" HeaderText="Title" />
 <asp:BoundField DataField="Author" HeaderText="Author" />
 <asp:BoundField DataField="Category" HeaderText="Category" />
 <asp:BoundField DataField="ISBN" HeaderText="ISBN" />
 <asp:BoundField DataField="Description" HeaderText="Description" />
</Columns>
</asp:GridView>
```

The noteworthy item is the <asp:Image tag.
The code: `ImageUrl='<%# (string) FormatImageUrl( (string) Eval("Image")) %>'` calls the FormatImageUrl method we just added to the cs file.

Do spend some time in Google looking at TemplateField and BoundField.

If you have loaded some images and the XML file, you should see the following:



Before you submit:

- Do not forget to set lstHidden.Visible to false.
- Be sure to comment grey using statements.
- Remember to indicate if you completed any extra credit.
- Be sure to copy the Img folder to the same level as App_Data and App_Code on the server.
- Be sure to add /Admin.aspx to the end of the URL or your program will report Access Forbidden.